FII Agent Manager API

Tecnical Design Document

Gabriel de Almeida Miki

Sumário

INTRODUCTION 03

- Business Requirements
  - Functional
  - Non Functional
  - Use Cases

TECHNICAL DESIGN 08

- Architectural Pattern
- Package Diagram
- Database Design

API SPECIFICATIONS 11

NEEDED LIBRARIES 13

DEVELOPMENT STEPS - TESTING 14

Introduction

The FII Portfolio Tracker is a containerized, agentic backend application designed to simplify the management and monitoring of Brazilian Real Estate Investment Funds (FIIs). Built with FastAPI and SQLModel, the platform exposes a RESTful API that enables full portfolio lifecycle management — from registering assets and logging transactions to refreshing live market prices. A Google ADK AI Agent provides natural language interaction for portfolio analysis, and an automated Notion sync keeps investors' dashboards up to date.

Functional

- Portfolio Management: Users must be able to add, view, update, and delete FII assets in their portfolio.
- Transaction Tracking: The system must record buy and sell transactions per asset, with quantity, price, and date.
- Market Data Refresh: Current prices for all assets must be refreshed via an asynchronous background task triggered on demand.
- Notion Sync: Users must be able to trigger a background synchronization of their portfolio to a linked Notion database.
- User Management: A user model must store email and Notion API credentials (API key and database ID).

Non Functional

- Asynchronous Processing: Background tasks (market refresh, Notion sync) must use FastAPI BackgroundTasks to avoid blocking the request-response cycle.
- Testability: All critical business logic and API endpoints must be covered by Pytest tests, including fixture-based database setup.
- Data Persistence: The PostgreSQL database must use Docker Volumes to survive container restarts without data loss.

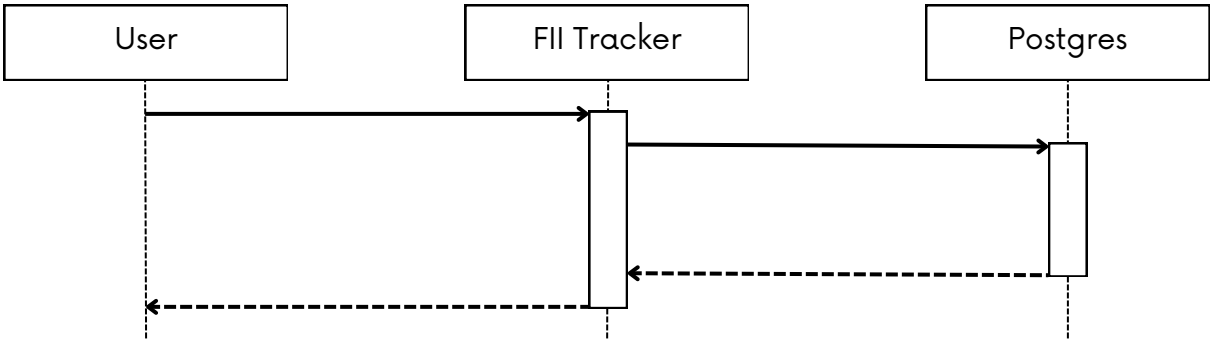
Use Case: Manage Portfolio Assets

- **Actors:**
  - **User:** The investor managing their FII portfolio.
  - **PostgreSQL Database:** Persistent store for assets and transactions.
- **Entry condition:**
  - The user is authenticated and calls one of the asset management endpoints.
- **Flow of Events:**
  - The user sends a POST /assets/assets/ request with symbol, name, sector, average buy price, quantity, and user ID.
  - The API validates the payload using the AssetCreate Pydantic schema.

Use Case: Manage Portfolio Assets

- The system persists the new asset in the PostgreSQL database and returns the created record.
- (Optional) The user sends a PUT /assets/assets/{asset_id} to update any field.
- (Optional) The user sends a DELETE /assets/assets/{asset_id} to remove the asset; all associated transactions are also deleted via cascade.
- The user queries GET /assets/portfolio/ to view the full portfolio with current values.

- **Exit condition:**
  - The asset is successfully created, updated, or deleted, and the response reflects the current state.
- **Exceptions:**
  - **Symbol Conflict:** If the ticker symbol already exists, the API returns 409 Conflict.
  - **Asset Not Found:** If the asset_id does not exist, the API returns 404 Not Found.
- **Special Requirements:**
  - Cascade delete must propagate to all linked transactions.
- **Nonfunctional Requirements, Constraints:**
  - All asset endpoints must respond within 300 ms under normal load.



Use Case: Refresh Market Data

- **Actors:**
  - **Admin / Scheduler**: The entity triggering the market data refresh.
  - **Background Task (FastAPI)**: The service executing the price update asynchronously.
  - **PostgreSQL Database**: Stores the updated current_price for each asset.
- **Entry condition:**
  - An authorized caller sends a POST /refresh request.
- **Flow of Events:**
  - The API immediately returns 202 Accepted to the caller.
  - FastAPI offloads the refresh logic to a BackgroundTask.
  - The background worker iterates over all assets in the database.

Use Case: Refresh Market Data

- For each asset, the worker fetches the current market price from an external data source.
- The worker updates the current_price field for each asset record.

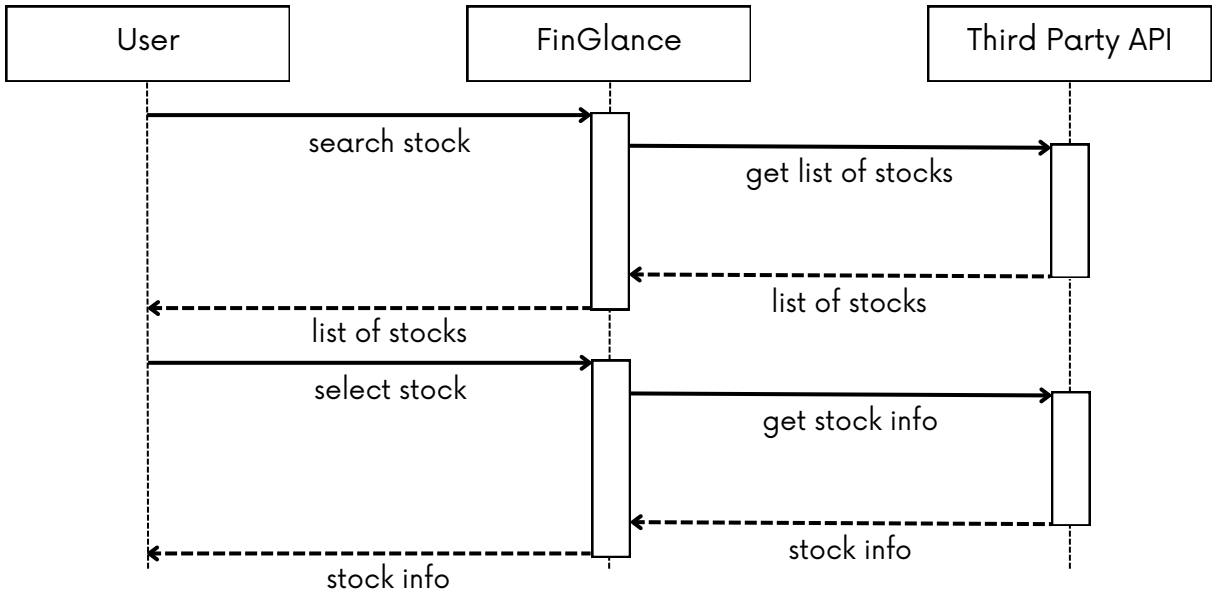
- **Exit condition:**
  - All asset current prices are updated in the database without blocking the API.
- **Exceptions:**
  - **External Source Unavailable**: If the market data provider is unreachable, the task logs the error and skips the affected assets.
- **Special Requirements:**
  - The endpoint must return 202 immediately; price fetching happens fully in the background.
- **Nonfunctional Requirements, Constraints:**
  - Market refresh must not block any other API operations while running.

Use Case: Sync Portfolio to Notion

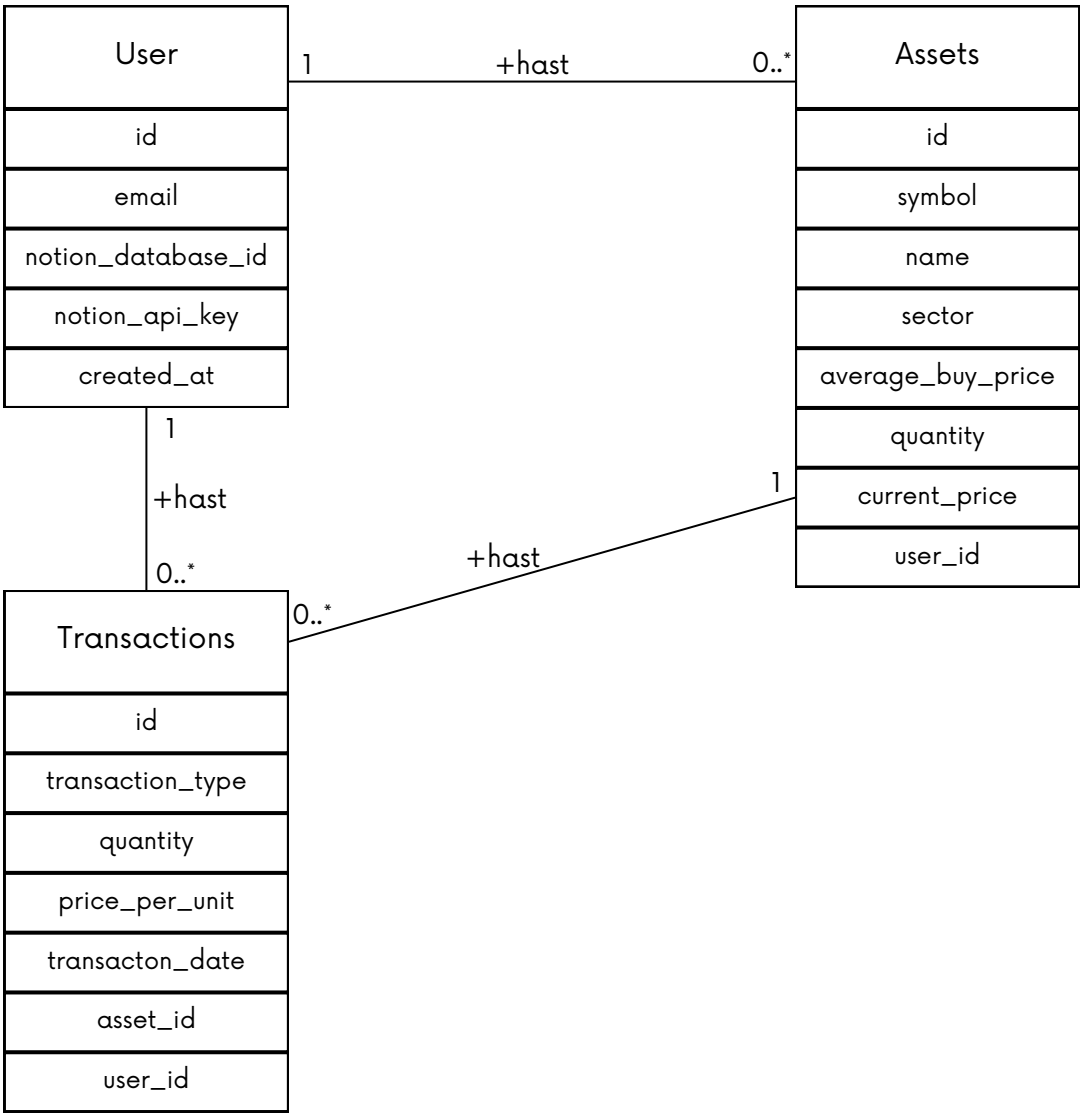
- **Actors:**
  - User: The investor who wants their portfolio reflected in Notion.
  - Background Task (FastAPI): The service executing the Notion synchronization.
  - Notion API: The external service receiving portfolio data.
- **Entry condition:**
  - The authenticated user sends a POST /sync/notion request.
- **Flow of Events:**
  - The API authenticates the caller via the get_current_user_simple dependency.
  - The API immediately returns 200 OK and offloads sync to a BackgroundTask.
  - The background task retrieves the user's Notion API key and database ID from the database.
  - The task fetches all assets associated with the user.
  - For each asset, the task upserts a row in the user's Notion database with current portfolio data.
- **Exit condition:**
  - The user's Notion database reflects the latest portfolio state.
- **Exceptions:**
  - Invalid Notion Credentials: If the Notion API rejects the key or database ID, the task logs the error and aborts the sync.
  - Notion Rate Limiting: If the Notion API throttles requests, the task implements exponential back-off.
- **Special Requirements:**
  - User Notion credentials must never be logged or exposed in error messages.
- **Nonfunctional Requirements, Constraints:**
  - The 200 OK response must be returned before the Notion sync completes.

Use Case: Manage Transactions

- **Actors:**
  - User: The investor recording buy/sell operations.
  - PostgreSQL Database: Persistent store for transaction records.
- **Entry condition:**
  - The user is authenticated and calls one of the transaction endpoints.
- **Flow of Events:**
  - The user sends a POST /transactions/ with transaction_type ('buy' or 'sell'), quantity, price_per_unit, transaction_date, and asset_id.
  - The API validates the payload and persists the transaction, linking it to the referenced asset.
  - (Optional) The user retrieves a single transaction via GET /transactions/{transaction_id}.
  - (Optional) The user retrieves all transactions via GET /transactions/.
  - (Optional) The user updates a transaction via PUT /transactions/{transaction_id}.
  - (Optional) The user deletes a transaction via DELETE /transactions/{transaction_id}.
- **Exit condition:**
  - The transaction is persisted and the user receives the full transaction object in the response.
- **Exceptions:**
  - Asset Not Found: If the asset_id does not reference an existing asset, the API returns 404 Not Found.
  - Transaction Not Found: If the transaction_id is invalid, the API returns 404 Not Found.
- **Nonfunctional Requirements, Constraints:**
  - Transaction endpoints must be idempotent for GET operations.

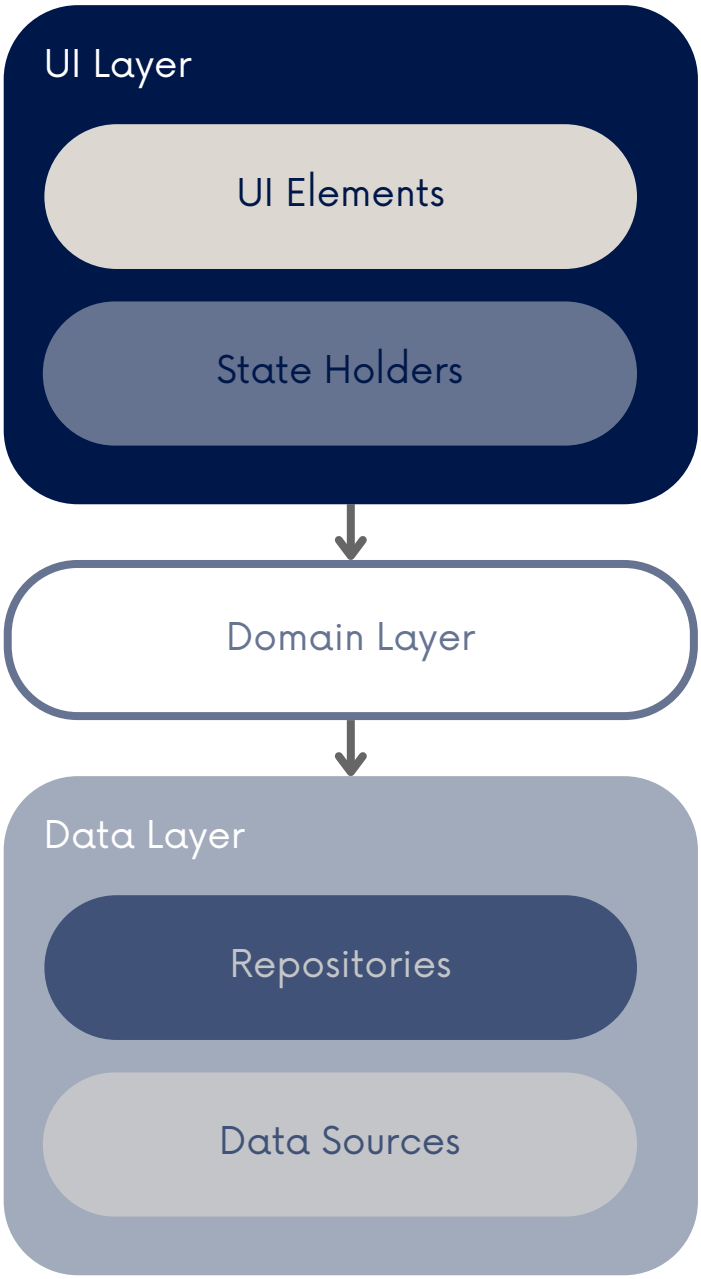


Database Design



Technical Design

The application is structured into three primary layers: the Interface (API) layer, the Domain (Business Logic) layer, and the Data and Infrastructure layer. This separation of concerns promotes maintainability, independent scalability of background workers, and testability across all components.



Interface Layer (API)

The API layer is built with FastAPI and acts as the entry point for all client requests. It is responsible for input validation via Pydantic/SQLModel schemas and for dispatching both synchronous responses and asynchronous background tasks. This layer comprises:

- **Routers / Endpoints:** Manage HTTP routes for assets, users, transactions, market refresh, and Notion sync. Each router is mounted with a dedicated prefix and tag.
- **Background Task Dispatcher:** Wraps long-running operations (market data refresh, Notion sync) using FastAPI's BackgroundTasks, returning an immediate HTTP response while work continues asynchronously.
- **Dependency Injection:** Provides database sessions (DatabaseSession) and authenticated user resolution (get_current_user_simple) to route handlers via FastAPI's Depends mechanism.

Domain Layer (Business Logic)

The domain layer houses the core business logic that distinguishes the application from a simple CRUD service. It remains decoupled from transport and persistence concerns. This layer comprises:

- Portfolio Service: Aggregates asset data, calculates total portfolio values using current prices, and orchestrates CRUD operations via the repository.
- Market Data Worker: Background task that iterates all assets, fetches live prices from an external data provider, and writes updates back to the database.
- Notion Sync Worker: Background task that serializes the user's portfolio into Notion page format and upserts entries to the user's linked Notion database via the Notion API.
- Google ADK AI Agent: Provides natural language interaction for portfolio analysis. The agent wraps domain tools (portfolio lookup, transaction history) and exposes them to an LLM reasoning loop.

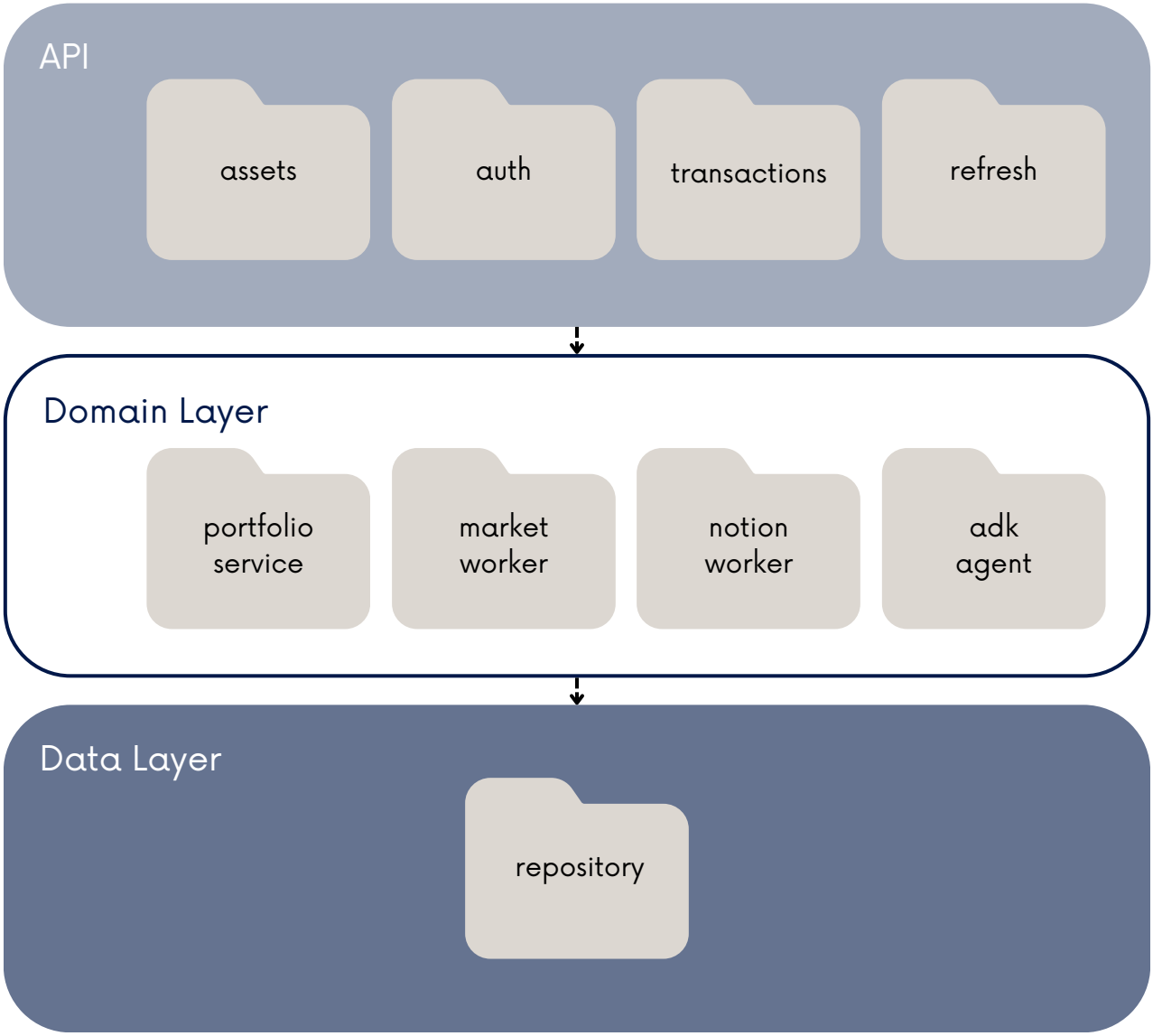
Data Layer (Persistence & Infrastructure)

The data layer manages all interactions with the PostgreSQL database and external integrations. It is composed of:

- SQLModel / SQLAlchemy Repository: Provides session-scoped database access, centralizing CRUD operations for User, Asset, and Transaction entities.
- PostgreSQL Container: The primary data store. Mounted with a Docker Volume to persist data across container lifecycles.
- Data Models (SQLModel): Unified schema definitions that serve as both database table mappings and Pydantic validation models, eliminating schema drift between layers.

Package Diagram

The package diagram below provides a high-level view of how the application is organized into distinct layers and modules, illustrating the dependency flow from the API layer down through the domain and into the data layer.



API Specification v1.0

Assets

Method	Endpoint	Status	Description
GET	/assets/portfolio/	200 OK	Retrieve all assets with totals and current values.
POST	/assets/assets/	201 Created	Add a new FII asset to the portfolio.
PUT	/assets/assets/{id}	200 OK	Update an existing asset's information.
DELETE	/assets/assets/{id}	204 No Content	Remove an asset and all linked transactions (cascade).

Users

Method	Endpoint	Status	Description
POST	/users/	201 Created	Create a new user with email and Notion credentials.
GET	/users/	200 OK	Retrieve a list of all users.
GET	/users/{user_id}	200 OK	Retrieve a specific user by UUID.
DELETE	/users/{user_id}	204 No Content	Delete a user and all their assets (cascade).

Transactions

Method	Endpoint	Status	Description
POST	/transactions/	201 Created	Record a new buy or sell transaction for an asset.
GET	/transactions/	200 OK	Retrieve all transactions.
GET	/transactions/{id}	200 OK	Retrieve a single transaction by UUID.
PUT	/transactions/{id}	200 OK	Update an existing transaction.
DELETE	/transactions/{id}	204 No Content	Delete a transaction by UUID.

Market Data

The /refresh endpoint returns 202 Accepted immediately. The background task iterates all assets in the database, fetches live market prices, and writes updated current_price values back to PostgreSQL. This prevents the request from blocking while potentially slow external calls are in flight.

Method	Endpoint	Status	Description
POST	/refresh	202 Accepted	Trigger a background task to refresh current prices for all assets.

Notion Sync

The /sync/notion endpoint resolves the current user from the auth dependency and immediately enqueues a background task. The task fetches the user's Notion credentials, retrieves all portfolio assets, and upserts each asset as a page in the linked Notion database.

Method	Endpoint	Status	Description
POST	/sync/notion	200 OK	Trigger a background sync of the authenticated user's portfolio to their

Needed Libraries

Core Framework

- fastapi: The primary web framework. Handles routing, dependency injection, and background task dispatch for all API endpoints.
- sqlmodel: Combines SQLAlchemy ORM with Pydantic validation. Provides a unified schema definition for both database tables and API request/response models.
- pydantic: Data validation and settings management. Automatically enforced on all request bodies and response models via SQLModel.

Application Server

- uvicorn: ASGI server that runs the FastAPI application. Supports async request handling and is required for FastAPI's background task mechanism.

Database

- psycopg2-binary: PostgreSQL adapter for Python. Provides the low-level driver that SQLAlchemy/SQLModel uses to connect to the containerized PostgreSQL instance.

AI & Agent

- google-adk: Google Agent Development Kit. Provides the agentic loop and tool-calling infrastructure for the natural language portfolio management agent.
- google-generativeai: Client library for Google's Gemini models. Used by the ADK agent for language understanding and reasoning.

Notion Integration

- notion-client: Official Notion Python SDK. Used by the Notion sync background worker to create and update pages in the user's linked Notion database.

Market Data

- httpx: Async HTTP client. Used by the market data worker to fetch live FII prices from external financial data providers.
- yfinance (optional): Yahoo Finance wrapper for fetching Brazilian FII prices by ticker symbol (e.g. HGLG11.SA).

Configuration & Security

- python-dotenv: Loads .env files into os.environ. Ensures all secrets (Notion keys, DB credentials, API keys) are managed via environment variables.

Testing

- pytest: Primary test runner. Fixtures handle database setup/teardown; all endpoints and background task logic are tested via the pytest suite.

Development Steps - Testing

The development process started in the database section. After the definition of which and how the data would be stored, the tests covering the definitions and basic functioning of the database were developed.

When you test a FastAPI application that uses a database, you face several challenges. You need a real database to test against because mocking database behavior is too complex and unreliable. But you also need each test to run in isolation - if one test creates data, you do not want that data affecting other tests. And critically, you want tests to run fast, because developers run tests frequently during development.

our testing strategy solves all these problems using a sophisticated pattern called "transaction rollback testing." The core idea is that you create the database and its tables once at the beginning of your test session, then for each individual test, you wrap all database operations in a transaction that gets rolled back when the test finishes. This means each test starts with a clean database state, but you avoid the massive overhead of creating and dropping tables repeatedly. This approach can make your test suite ten or even a hundred times faster than simpler strategies.

Stage One: Session Setup - Creating the Test Database

```python
@pytest.fixture(scope="session", autouse=True)
def setup_test_database():
    """
    Create test database once before all tests.
    Tables are created once and reused with transaction rollback.
    """
    if not database_exists(TEST_DATABASE_URL):
        print(f"\n📦 Creating test database...")
        create_database(TEST_DATABASE_URL)

    engine = create_engine(TEST_DATABASE_URL)

    # Create all tables once
    print(f"🔧 Creating tables...")
    Base.metadata.create_all(bind=engine)

    yield

    # Cleanup after all tests
    print(f"\n🧹 Dropping tables...")
    Base.metadata.drop_all(bind=engine)
    engine.dispose()
```

When pytest starts, it looks for fixtures marked with scope="session" and autouse=True. Your setup_test_database fixture has both of these markers, which means pytest runs it once before any tests execute, automatically without any test needing to request it explicitly. The next one (seen bellow) is used to create the engine that will manage all tests execution.

The first thing this fixture does is check if your test database exists using database_exists. This is checking if a PostgreSQL database called test_mydatabase exists on your database server. If it does not exist, the code creates it with create_database. This is important because you cannot create tables in a database that does not exist yet. The database itself is just a container - the actual structure comes from the tables you create inside it.

Next, the fixture creates a SQLAlchemy engine that connects to this test database. The engine is SQLAlchemy's way of managing the connection pool and translating Python code into SQL for your specific database. The TEST_DATABASE_URL tells it to connect to PostgreSQL running on localhost with the credentials myuser:mypassword and to use the test_mydatabase database.

Then comes the crucial line: Base.metadata.create_all(bind=engine). This is where SQLAlchemy reads all your model definitions (all the classes that inherit from Base, like your User, Asset, and Transaction models) and generates CREATE TABLE statements for each one. It executes these statements against your test database, creating the entire schema - all tables, columns, foreign keys, indexes, and constraints. This happens once for the entire test session, not for each test, which is a major performance optimization.
The yield keyword is where the fixture pauses and lets all your tests run. The fixture is essentially saying "I have set up the database, now you can run all your tests, and when you are done, come back here." After all tests complete, execution resumes after the yield, and the fixture cleans up by dropping all tables and disposing of the engine. This ensures your test database is left clean for the next test run.

Stage Two: Creating the Shared Engine

The next fixture pytest sets up is db_engine, also with scope="session":

```
@pytest.fixture(scope="session")
def db_engine():
    """Database engine shared across all tests."""
    engine = create_engine(
        TEST_DATABASE_URL,
        pool_pre_ping=True,
        echo=False  # Set to True to see SQL queries
    )
    yield engine
    engine.dispose()
```

This creates another engine to the same test database, but this engine is what your tests will actually use. You might wonder why you need two engines - one in setup_test_database and one here. The reason is separation of concerns. The first engine is for one-time setup and teardown operations. This second engine is configured specifically for testing with pool_pre_ping=True, which makes SQLAlchemy test connections before using them to catch stale connections. The echo=False parameter controls whether SQLAlchemy prints all SQL statements to the console, which you can set to True when debugging.

This engine fixture yields the engine object and then waits. All tests that need database access will use this same engine throughout the test session. At the very end, after all tests finish, it disposes of the engine properly, closing all connections.

Stage Three: The Magic of Transaction Rollback

Now we get to the most sophisticated part of your testing strategy - the test_db fixture. This fixture has scope="function", which means pytest creates a fresh instance for every single test function. This is where the isolation between tests happens. Let me walk through this fixture line by line because it is doing some advanced SQLAlchemy work:

```
@pytest.fixture(scope="function")
def test_db(db_engine):
    """
    Provides a database session that automatically rolls back after each test.
    This is much faster than creating/dropping tables for each test.
    """
    connection = db_engine.connect()
    transaction = connection.begin()

    session = sessionmaker(bind=connection)()

    # Start a SAVEPOINT (nested transaction)
    nested = connection.begin_nested()

    # Each time the SAVEPOINT ends, reopen it
    @event.listens_for(session, "after_transaction_end")
    def end_savepoint(session, transaction):
        nonlocal nested
        if not nested.is_active:
            nested = connection.begin_nested()

    yield session

    # Rollback the overall transaction, restoring database to clean state
    session.close()
    transaction.rollback()
    connection.close()
```

First, it gets a connection from the shared engine using db_engine.connect(). This pulls a connection from the connection pool. Then immediately, it starts a transaction with connection.begin(). This is critical - from this point forward, all database operations on this connection happen inside a transaction. Nothing is permanently committed to the database yet.

Next, it creates a SQLAlchemy session bound to this connection. The session is what your test code will use to query and modify the database. By binding it to the connection that already has an active transaction, all the session's operations participate in that transaction.

Here is where it gets really clever. The code starts a nested transaction with connection.begin_nested(). In PostgreSQL, nested transactions are implemented as SAVEPOINTs. A SAVEPOINT is like a bookmark in the transaction - you can roll back to a SAVEPOINT without rolling back the entire transaction. This is important because your test code might call session.commit(), which would normally commit the transaction and persist changes to the database. But with a SAVEPOINT, when your test calls session.commit(), it only commits to the SAVEPOINT, not to the database. The changes stay in the transaction and get rolled back when the test finishes.

The event listener is there to automatically recreate the SAVEPOINT after each commit. When your test calls session.commit(), SQLAlchemy triggers the "after_transaction_end" event. The listener checks if the SAVEPOINT is no longer active (because it was committed), and if so, it immediately creates a new SAVEPOINT. This means your test can call commit multiple times, and each time it commits to a fresh SAVEPOINT. The changes appear committed from the test's perspective, but they are still inside the outer transaction that will be rolled back.

After yielding the session to the test, when the test finishes, execution continues and the fixture performs cleanup. It closes the session, then critically calls transaction.rollback(). This rolls back the entire transaction, including all the commits to SAVEPOINTs that happened during the test. All database changes made by the test are discarded. Finally, it closes the connection, returning it to the pool.

The beauty of this approach is that from your test's perspective, it has a normal database session that it can commit to and query from. But in reality, nothing is permanently written to the database. Each test starts with the exact same empty tables that were created at the beginning of the session.

Stage Four: Integrating FastAPI

The client fixture brings FastAPI into the picture, creating a test client that uses your isolated database session:

```python
@pytest.fixture(scope="function")
def client(test_db):
    """FastAPI test client using test database."""
    def override_get_db():
        try:
            yield test_db
        finally:
            pass

    app.dependency_overrides[get_db] = override_get_db

    with TestClient(app) as test_client:
        yield test_client

    app.dependency_overrides.clear()
```

This fixture depends on test_db, which means pytest will create a fresh test_db session first, then pass it into this fixture. The fixture creates a function called override_get_db that yields the test database session. This is the key to making FastAPI use your test database instead of the real database.

FastAPI's dependency injection system has a feature called dependency_overrides that lets you replace dependencies during testing. Your FastAPI app normally uses a get_db dependency that yields a database session from the production database. By setting app.dependency_overrides[get_db] = override_get_db, you are telling FastAPI "whenever you need a database session, use my test session instead." This means when your FastAPI endpoints query the database, they query the test database with the transaction that will be rolled back.

The TestClient is FastAPI's testing tool that lets you make HTTP requests to your application without actually running a server. It calls your endpoints directly in the same process, which makes tests fast. The context manager ensures that after the test finishes, the test client is properly closed.

Finally, app.dependency_overrides.clear() removes the override, resetting the application to its normal state. This is important because pytest might reuse the application object across tests, and you want each test to start fresh.

Stage Five: Test-Specific Fixtures

Your test_user fixture shows a pattern for creating test data:

```python
@pytest.fixture
def test_user(test_db):
    """Provides a test user for tests"""
    from app.db import User
    from uuid import uuid4

    user = User(
        id=uuid4(),
        email="test@example.com",
        notion_database_id="test-notion-db",
        notion_api_key="test-api-key"
    )
    test_db.add(user)
    test_db.commit()  # This commits to the SAVEPOINT
    test_db.expire_all()  # Force reload from transaction

    # Re-query to ensure it's attached to session
    user = test_db.query(User).filter(User.email == "test@example.com").first()

    return user
```

This fixture creates a User object and adds it to the test database session. When it calls test_db.commit(), remember that this commits to the SAVEPOINT, not to the actual database. The user exists in the transaction for the duration of the test, but will be rolled back when the test finishes.

The test_db.expire_all() line tells SQLAlchemy to forget the cached state of all objects, forcing them to be reloaded from the database on next access. This ensures you are working with data as it actually exists in the transaction, not stale cached values. The re-query at the end ensures the user object is properly attached to the session, which is important because SQLAlchemy sessions can be finicky about object states.

Any test that needs a user can simply accept the test_user fixture as a parameter, and pytest will automatically create the user, run the test, then roll back the user's creation when the test finishes.

How Tests Use This Infrastructure

Looking at your test_database_config.py file, you can see how tests actually use these fixtures. Let me trace through one test to show the complete flow:

```python
def test_tables_creation(test_db):
    """
    Test that all tables defined in models are created correctly.
    This verifies Base.metadata.create_all() works.
    """
    inspector = inspect(test_db.bind)
    table_names = inspector.get_table_names()

    assert "assets" in table_names
    assert "transactions" in table_names
    assert "users" in table_names
```

When pytest encounters this test function, it sees that the function accepts a parameter called test_db. Pytest looks for a fixture with that name and finds your test_db fixture. Before running the test, pytest runs the entire fixture chain: it ensures the session-level setup_test_database has run (creating the database and tables), it ensures the db_engine exists, then it creates a fresh test_db session with its transaction and SAVEPOINT. Only then does pytest call your test function, passing in the session.

Inside the test, you create a SQLAlchemy inspector to examine the database schema. The inspector is bound to test_db.bind, which is the connection with the active transaction. When you call inspector.get_table_names(), it queries the database schema and returns the list of tables. Your assertions check that the expected tables exist, verifying that the setup fixture correctly created them.

When the test finishes, pytest runs the teardown portion of the test_db fixture. It rolls back the transaction (though this test did not modify anything), closes the session and connection, and returns them to the pool. The tables still exist because they were created at the session level, but any data changes would have been rolled back.

The Complete Flow Visualized

First, pytest discovers all test files and collects all test functions. It analyzes which fixtures each test needs and builds a dependency graph. It sees that some fixtures have session scope, some have function scope, and some fixtures depend on others.

Second, pytest executes all session-scoped fixtures with autouse=True. Your setup_test_database fixture runs, creating the database if needed and creating all tables. This happens once, before any tests run.

Third, pytest creates session-scoped fixtures that tests depend on. Your db_engine fixture runs, creating the engine that will be shared across all tests.

Fourth, for each test function, pytest creates the function-scoped fixtures the test needs. If the test accepts test_db, pytest runs that fixture, which creates a connection, starts a transaction, creates a session with a SAVEPOINT, and yields the session to the test. If the test accepts client, pytest runs the client fixture, which depends on test_db, so test_db runs first, then client creates the test client with the overridden database dependency.

Fifth, pytest runs the actual test function, passing in the fixtures. The test code executes, making API calls through the test client or querying the database directly through the test session. All database operations happen inside the transaction.

Sixth, when the test finishes, pytest runs the teardown portion of the fixtures in reverse order. The client fixture clears the dependency override. The test_db fixture rolls back the transaction, discarding all changes, then closes the session and connection. The test is complete.

Seventh, pytest repeats steps four through six for each test function. Each test gets a fresh transaction with empty tables, runs in isolation, and has its changes rolled back.